

Inside Lemma31.lean

A Picture Book of One Clean Trick

Abstract

This note opens up one file: `Lemma31.lean`.

It proves **Lemma 3.1** of Dempwolff & Müller. The lemma says:

$$x \mapsto L(x) \cdot M(x) \text{ injective} \iff x \mapsto L^*(x) \cdot M^{-1}(x) \text{ injective.}$$

Here L is linear, M is multiplicative, and L^* is the *trace-adjoint* of L .

We walk through every block in the file. Each block gets a small picture, the matching Lean snippet, and the Mathlib pieces it leans on. The English is kept plain on purpose.

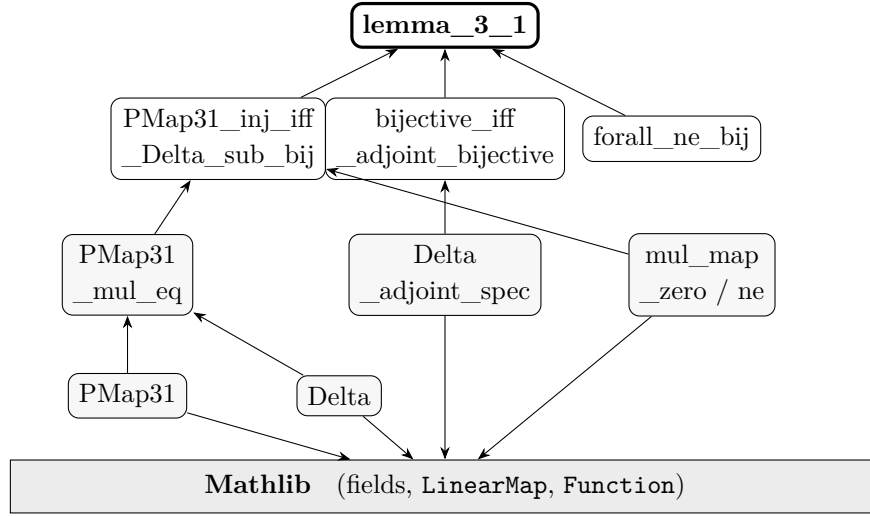
1 The shape of the file

The file builds one big equivalence out of small steps.

The pieces, bottom to top:

- `PMap31` — the product map $P = L \cdot M$.
- `Delta` — a linear “twist” map Δ_y .
- `mul_map_zero`, `mul_map_ne_zero` — M behaves at 0.
- `PMap31_mul_eq` — factor $P(xy)$ through Δ_y .
- `PMap31_inj_iff_Delta_sub_bij` — product injective $\Leftrightarrow$ differences bijective.
- `bijective_iff_adjoint_bijective` — a map is bijective $\Leftrightarrow$ its adjoint is.
- `Delta_adjoint_spec` — the adjoint of Δ is again a Δ .
- `forall_ne_bij` — relabel “for all $y_1 \neq y_2$ ” through a bijection.

- lemma_3_1 — glue the pieces into the final equivalence.



The whole file imports *only* Mathlib. So every block sits straight on top of Mathlib.

The setting is fixed once for the file:

```
-- RequestProject/FiniteField/Lemma31.lean
variable {K F : Type*} [Field K] [Field F] [Algebra K F]
[FiniteDimensional K F] [Finite F]
```

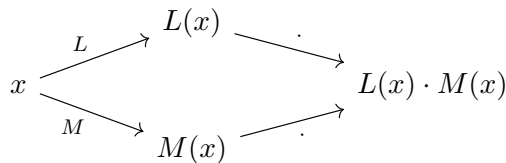
Mathlib blocks here:

- Field — K and F are fields.
- Algebra $K\ F$ — F is a vector space (and algebra) over K .
- FiniteDimensional, Finite — so injective = surjective.

2 Block 1 — PMap31: the product map

The whole story is about one map. Take a linear map L and any map M . Multiply their outputs:

$$P(x) = L(x) \cdot M(x).$$



```
def PMap31 (L : F →1[K] F) (M : F → F) (x : F) : F := L x * M x
```

Mathlib blocks: the bundled K -linear map type $F \rightarrow_\ell F$, and field multiplication.

3 Block 2 — Delta: the twist map

Now fix a second input y . Twist x by y *before* applying L , then scale by $M(y)$:

$$\Delta_y(x) = L(x \cdot y) \cdot M(y).$$

The key point: for a fixed y , this is **linear in x** . So Δ_y is a genuine `LinearMap`.

$$x \xrightarrow{\cdot y} xy \xrightarrow{L} L(xy) \xrightarrow{\cdot M(y)} \Delta_y(x)$$

```
noncomputable def Delta (L : F →1[K] F) (M : F → F) (y : F) : F →1[K] F where
  toFun x := L (x * y) * M y
  map_add' ₁ x ₂ x := by simp [add_mul, map_add, add_mul]
  map_smul' r x := by
    show L ((r • x) * y) * M y = r • (L (x * y) * M y)
    rw [smul_mul_assoc, map_smul, smul_mul_assoc]

@[simp] lemma Delta_apply (L : F →1[K] F) (M : F → F) (y x : F) :
  Delta L M y x = L (x * y) * M y := rfl
```

Proof steps:

- `map_add'`: opens $(x_1 + x_2)y$ with `add_mul`, then L is additive (`map_add`).
- `map_smul'`: scalars slide out with `smul_mul_assoc` and L is linear (`map_smul`).
- `Delta_apply`: holds by `rfl` (just unfolding).

Mathlib blocks: `add_mul`, `map_add`, `map_smul`, `smul_mul_assoc`, and the `LinearMap` bundle (`toFun`, `map_add'`, `map_smul'`).

4 Block 3 — M behaves at zero

We need two tiny facts about a multiplicative, injective M .

- If $M(ab) = M(a)M(b)$ and M is injective, then $M(0) = 0$.

- Then M sends nonzero to nonzero.

Why $M(0) = 0$? If not, then $M(0) = M(0)M(x)$ forces $M(x) = 1$ for *every* x . That kills injectivity. Contradiction.

$$M(0) \neq 0 \xrightarrow{M(0) = M(0)M(x)} M \equiv 1 \xrightarrow{M(0) = M(1)} \text{not injective}$$

```

lemma mul_map_zero {M : F' → F'} (hM : ∀ a b, M (a * b) = M a * M b)
  (hMinj : Function.Injective M) : M 0 = 0 := by
  by_contra h
  have hM0 : ∀ x : F', M x = 1 := by intro x; specialize hM 0 x; aesop
  simp [hM0] using @hMinj 0 1

lemma mul_map_ne_zero {M : F' → F'} (hMinj : Function.Injective M)
  (hM0 : M 0 = 0) {x : F'} (hx : x ≠ 0) : M x ≠ 0 :=
  fun h => hx (hMinj (h.trans hM0.symm))

```

Proof steps: `by_contra`, then `aesop` closes the “ $M \equiv 1$ ” step; `simp` hits injectivity at 0 vs 1.

Mathlib blocks: `Function.Injective`, `by_contra`, `aesop`, `Eq.trans`.

5 Block 4 — PMap31_mul_eq: factor the product

This is the bridge between P and Δ .

Feed P the product $x \cdot y$. Then it splits:

$$P(xy) = \Delta_y(x) \cdot M(x).$$

So the product map, evaluated on xy , is the twist map times $M(x)$.

$$P(xy) = \Delta_y(x) \cdot M(x)$$

```

lemma PMap31_mul_eq (L : F →1 [K] F) {M : F → F}
  (hM : ∀ a b, M (a * b) = M a * M b) (x y : F) :
  PMap31 L M (x * y) = Delta L M y x * M x := by
  simp [PMap31, hM]; ring

```

Proof steps: unfold `PMap31`, split $M(xy) = M(x)M(y)$ with `hM`, then `ring` rearranges the product.

Mathlib blocks: simp, ring.

6 Block 5 — product injective $\Leftrightarrow$ differences bijective

This is the heart of the file.

Claim. P is injective exactly when, for every $y_1 \neq y_2$, the difference of twist maps

$$\Delta_{y_1} - \Delta_{y_2}$$

is bijective.

$$\boxed{P = L \cdot M \text{ injective}} \iff \boxed{\begin{array}{c} \Delta_{y_1} - \Delta_{y_2} \text{ bijective} \\ \text{for all } y_1 \neq y_2 \end{array}}$$

Why it works, in one breath: $P(xy_1) = P(xy_2)$ becomes $(\Delta_{y_1} - \Delta_{y_2})(x) = 0$ after the factor trick (Block 4). So “no collisions” for P means “trivial kernel” for the difference — i.e. injective, hence bijective in finite dimension.

```
lemma PMap31_inj_iff_Delta_sub_bij (L : F →1 [K] F) {M : F → F}
  (hM : ∀ a b, M (a * b) = M a * M b) (hMinj : Function.Injective M) :
  Function.Injective (PMap31 L M) ↔
  ∀ 1y 2y : F, 1y ≠ 2y → Function.Bijective (Delta L M 1y - Delta L M 2y)
```

Proof steps:

- ($\Rightarrow$) From a kernel vector x , build $P(xy_1) = P(xy_2)$, apply injectivity, get $x = 0$; so the kernel is trivial.
- Trivial kernel $\Rightarrow$ injective via `LinearMap.ker_eq_bot`; then injective $\Rightarrow$ surjective via `LinearMap.surjective_of_injective`.
- ($\Leftarrow$) If P collides at x, y with $x \neq y$, the difference map kills 1, contradicting bijectivity.

Mathlib blocks: `LinearMap.ker_eq_bot`, `LinearMap.ker_eq_bot'`, `LinearMap.surjective_of_injective`, `Function.Bijective`.

7 Block 6 — a map is bijective iff its adjoint is

Now bring in a nondegenerate *trace form* $T : F \rightarrow K$. “Nondegenerate” means: if $T(xy) = 0$ for all y , then $x = 0$.

Suppose A and A^* are adjoint with respect to T :

$$T(Au \cdot v) = T(u \cdot A^*v) \quad \text{for all } u, v.$$

Then A is bijective *iff* A^* is bijective.

$$T(Au \cdot v) = T(u \cdot A^*v) \implies (A \text{ bijective}) \Leftrightarrow (A^* \text{ bijective}).$$

```

Lemma bijective_iff_adjoint_bijective (T : F →1[K] K)
  (hT : ∀ x : F, ∀ ( y : F, T (x * y) = 0) → x = 0)
  (A Aadj : F →1[K] F)
  (hAdj : ∀ u v, T (A u * v) = T (u * Aadj v)) :
  Function.Bijective A ↔ Function.Bijective Aadj

```

Proof steps:

- One direction: surjectivity of A plus the pairing makes $T(\cdot, \cdot)$ nondegenerate enough to force A^* injective.
- Injective $\Rightarrow$ bijective by finiteness (`Finite.injective_iff_surjective`).
- The reverse direction is the mirror image.

Mathlib blocks: `sub_eq_zero`, `mul_sub`, `sub_mul`, `Finite.injective_iff_surjective`, `LinearMap.surjective_of_injective`.

8 Block 7 — the adjoint of a twist is a twist

We need the adjoint of Δ to stay in the same family.

With L^* the adjoint of L and M^{-1} the inverse of M :

$$T(\Delta_y^{L,M}(u) \cdot v) = T(u \cdot \Delta_{M(y)}^{L^*,M^{-1}}(v)).$$

So the adjoint of “twist by y with (L, M) ” is again a twist — now “twist by $M(y)$ with (L^*, M^{-1}) ”. The family is closed under adjoints.

$$\Delta_y^{L,M} \xrightarrow{\text{adjoint}} \Delta_{M(y)}^{L^*,M^{-1}}$$

```

Lemma Delta_adjoint_spec (T : F →1[K] K) (L Ladj : F →1[K] F)

```

```

(hAdj : ∀ w z, T (L w * z) = T (w * Ladj z))
{M Minv : F → F} (hMinv : ∀ x, Minv (M x) = x)
(u v y : F) :
  T (Delta L M y u * v) = T (u * Delta Ladj Minv (M y) v) := by
have := hAdj (u * y) (v * M y)
simp_all +decide [mul_assoc, mul_left_comm]
convert this using 1 <=> ring

```

Proof steps: feed the L/L^* adjoint rule the points $(uy, v M(y))$, simplify the products, then `ring` matches both sides.

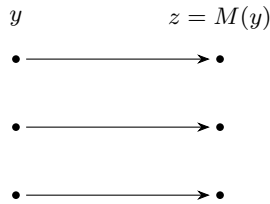
Mathlib blocks: `mul_assoc`, `mul_left_comm`, `convert`, `ring`, `simp_all`.

9 Block 8 — relabel “for all $y_1 \neq y_2$ ”

A small combinatorial step.

We have a property Q stated for all pairs $M(y_1) \neq M(y_2)$. Since M is a bijection, this is the same as Q for all pairs $z_1 \neq z_2$.

In words: a bijection just *renames* the elements, so a “for all distinct pairs” statement is unchanged.



distinct pairs $\leftrightarrow$ distinct pairs

```

Lemma forall_ne_bij α{ : Type*} {M : α → α} (hMbij : Function.Bijective M)
  {Q : α → α → Prop} :
  ∀( ₁y ₂y, ₁y ≠ ₂y → Q (M ₁y) (M ₂y)) ↔ ∀( ₁z ₂z, ₁z ≠ ₂z → Q ₁z ₂z) := by
constructor <=> intro h ₁z ₂z hz
  · obtain {₁y, }rfl := hMbij.2 ₁z
    obtain {₂y, }rfl := hMbij.2 ₂z
    exact h ₁y ₂y (fun heq => hz (congr_arg M heq))
  · exact h _ _ (fun heq => hz (hMbij.1 heq))

```

Proof steps: pull preimages with surjectivity (`hMbij.2`); keep distinctness with injectivity

(hMbij.1).

Mathlib blocks: `Function.Bijective`, `congr_arg`.

10 Block 9 — lemma_3_1: glue it all

Finally, put the blocks together.

Lemma 3.1.

$$L(x) \cdot M(x) \text{ injective} \iff L^*(x) \cdot M^{-1}(x) \text{ injective.}$$

The chain of rewrites:

1. Rewrite *both* sides with Block 5 (`PMap31_inj_iff_Delta_sub_bij`):
each side becomes “all differences bijective”.
2. For each pair y_1, y_2 , use Block 6 (`bijective_iff_adjoint_bijective`) with the adjoint from Block 7 (`Delta_adjoint_spec`):
the (L, M) difference is bijective iff the (L^*, M^{-1}) difference is.
3. The right side ranges over $M(y_1) \neq M(y_2)$. Block 8 (`forall_ne_bij`) relabels it to all $z_1 \neq z_2$.

$$L \cdot M \text{ inj} \xrightarrow{\text{Block 5}} \Delta^{L, M} \text{ bij} \xrightarrow{\text{Blocks 6,7}} \Delta^{L^*, M^{-1}} \text{ bij} \xrightarrow{\text{Block 8}} L^* \cdot M^{-1} \text{ inj}$$

```

theorem lemma_3_1 (T : F →1 [K] K)
  (hT : ∀ x : F, ∀ (y : F, T (x * y) = 0) → x = 0)
  (L Ladj : F →1 [K] F)
  (hAdj : ∀ w z, T (L w * z) = T (w * Ladj z))
  (M Minv : F → F)
  (hM_mul : ∀ a b, M (a * b) = M a * M b)
  (hM_bij : Function.Bijective M)
  (hMinv_mul : ∀ a b, Minv (a * b) = Minv a * Minv b)
  (hMinv_inj : Function.Injective Minv)
  (hMinvL : ∀ x, Minv (M x) = x)
  (hMinvR : ∀ x, M (Minv x) = x) :
  Function.Injective (PMap31 L M) ↔ Function.Injective (PMap31 Ladj Minv) := by
  rw [PMap31_inj_iff_Delta_sub_bij L hM_mul hM_bij.injective]
  rw [PMap31_inj_iff_Delta_sub_bij Ladj hMinv_mul hMinv_inj]
  have h_bijective_iff_adjoint_bijective :
    ∀ 1y 2y : F, Function.Bijective (Delta L M 1y - Delta L M 2y) ↔
      Function.Bijective (Delta Ladj Minv (M 1y) - Delta Ladj Minv (M 2y)) := by
  intro 1y 2y

```



```

    apply bijective_iff_adjoint_bijective T hT
  intro u v; simp +decide [Delta, hAdj]; ring
  simp +decide [mul_assoc, hAdj, hMinvL]
  simp +decide only [mul_comm]
  convert forall_ne_bij hM_bij using 1
  simp +decide only [h_bijective_iff_adjoint_bijective]

```

Mathlib blocks: `rw`, `convert`, `simp`, `ring`, plus the file’s own Blocks 5–8.

11 One sentence to remember

`Lemma31.lean` swaps a hard *product* question for an easy *linear* one — and a trace adjoint flips the linear map and inverts the multiplier, leaving injectivity untouched.

Two maps, one twist, one adjoint, one equivalence.